Prelab 06: Datapath Components I

Started: Oct 13 at 9:56am

Quiz Instructions

Objective

In this lab you will implement various datapath elements, including the sign extender, ALU and NextPC modules. You are expected to be familiar with ALU design (see Appendix A.5 of the textbook). In addition, you should have a thorough understanding of the ARMv8 instruction set.

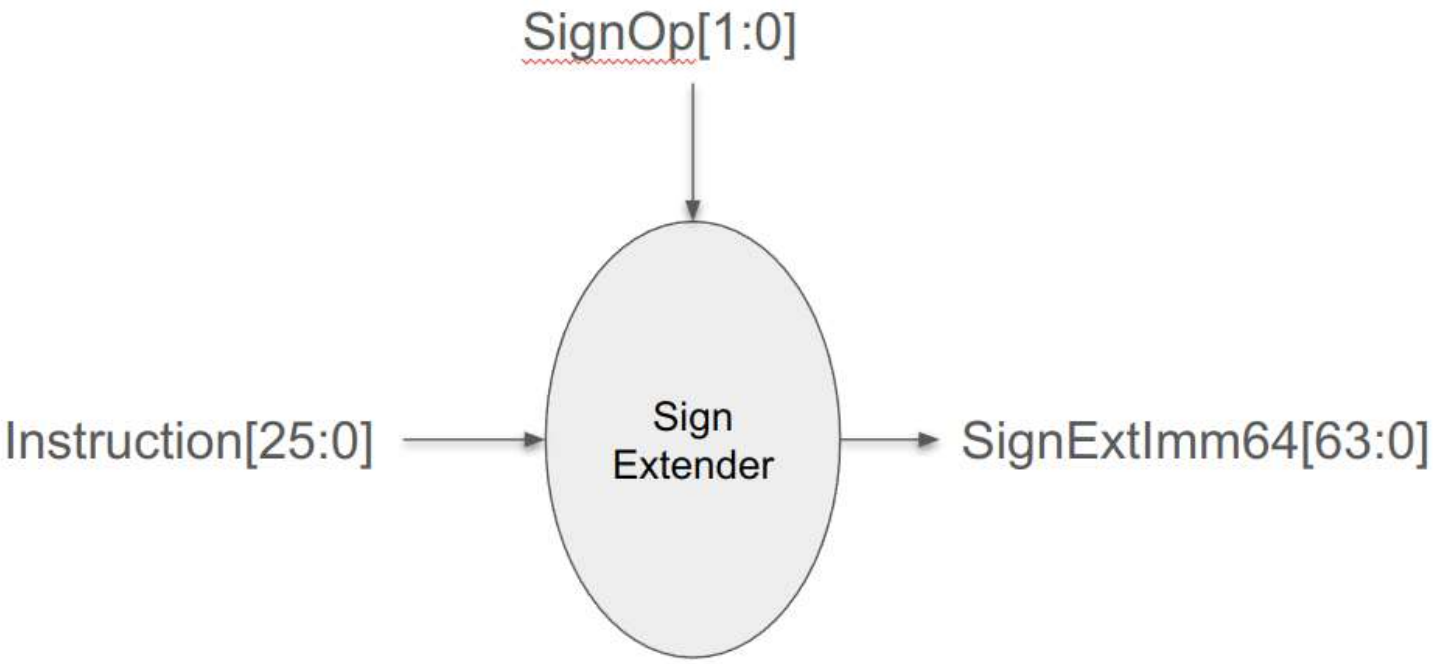
Datapath Elements

In this lab, you will design the 64-bit Arithmetic Logic Unit (ALU) as described in Appendix A.5 of the textbook and the Sign Extender unit. These are important parts of the ARMv8 microprocessor you will build in later labs, so it is advised to do this prelab and lab carefully!

Sign Extender Block

The sign extender in your ARMv8 processor makes sure that immediate values encoded in the instructions are extended to 64-bits so that they can be used in the ALU by immediate arithmetic (I-Type) and load/store instructions (D-Type), or used to calculate the next PC for branch instructions(CB- or B-Type) . Since each of these instruction types has a different sized immediate field, the "SignOp" input of the of the inputs of the unit will inform the unit what type of instruction is being extended and hence which bits from the instruction to use (and whether to sign-extend or zero-extend the value).

Here is a block diagram for this component:

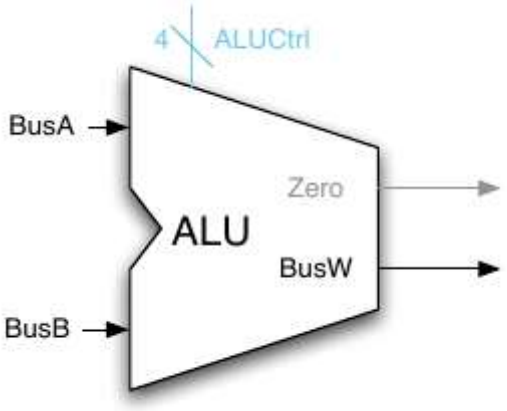


For this prelab and lab, assume that the SignOp field can have the following values corresponding the types of instructions that must be extended:

SignOp	Instruction Type
00	I-Type
01	D-Type
10	CB-Type
11	B-Type

ALU Block

As you know, the ALU is the most crucial element in any processor datapath. Almost every instruction uses ALU for some processing. For example, ADD uses the ALU for addition, SUB for subtraction, ORR for logical OR, etc. As shown in the Figure below, an ALU module takes two 64-bit inputs (BusA and BusB), performs an operation as specified by the 4-bit ALUCtrl field, and outputs the result on the 64 bit output bus BusW. When the value on BusW is 0, the Zero signal is set to 1, otherwise it is set to 0.



The basic operations performed by an ALU that you must support are:

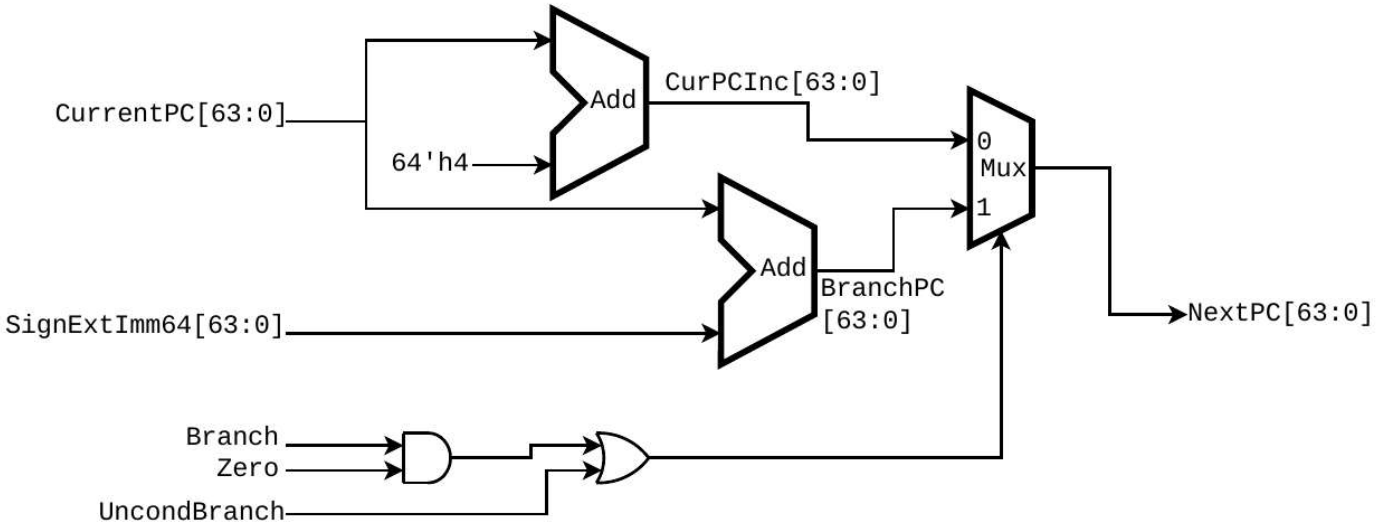
1. Addition
2. Subtraction
3. OR
4. AND
5. Pass input B (used by CBZ)

The control sequences are given by the following table.

Operation	ALU Control Line
AND	0000
OR	0001
ADD	0010
SUB	0110
PassB	0111

NextPC logic

The NextPC logic has the task of generating the next PC. Based on whether the instruction is a branch or not it selects the sign extended immediate or the current PC as the next PC value. Here is a small diagram showing the basic structure of the NextPC block.



Question 1 20 pts

The sign extender you are designing will have to either sign or zero extend a different set of bits from the instruction depending on the instruction type. Fill in the blanks in the table below accordingly.

Instruction Type	SignOp[1:0] Encoding	Bits of Instruction needed	Sign or Zero extend (S/Z)	Left shift output by 2 bits? (Y/N)
I-Type	00	[21 : 10]	Z	N
D-Type	01	[20 : 12]	S	N
CB-Type	10	[23 : 5]	S	Y
B-Type	11	[25 : 0]	S	Y

Question 2 30 pts

Write the Verilog implementation of the Sign Extender as specified in the instructions section at the top of this prelab. In particular, your implementation should conform to the following module definition:

```
`define Itype   2'b00  // These defines are not required but may be helpful.
`define Dtype    2'b01
`define CBtype   2'b10
`define Btype   2'b11

module SignExtender(
    output reg [63:0] SignExOut,
    input      [25:0] Instruction,
    input      [1:0]  SignOp
);
```

(This module definition can be found in your Lab06 director as well)

Your Verilog implementation should conform to the following characteristics:

- It should be written in fully synthesizable behavioral Verilog with no unintended latches.
- Assume that SignOp signal is encoded as described in the instructions for this prelab above.
- Your code should perform either zero or sign extension as described in the previous problem.
- For the instruction types where the Immediate must be left shifted by two bits, this should be done with two 0 bits shifted in from the right.
- Copy your Verilog code into the space below.

You may leverage an LLM in generating this code, however you must ensure it does not have bugs and you will have to explain how it works during Lab.

Note: here again, I found LLMs produced buggy output, be wary if you want to receive full credit.

Edit View Insert Format Tools Table

12pt Paragraph B I U A T²

p 0 words </>

Question 3 30 pts

Write a self-checking testbench for your Sign Extender.

Again, we will use a limited random testing strategy as we did in the previous lab. Thus, your testbench should perform tests which conform to the following constraints:

- The testbench should be self-checking, printing an error for each test that fails.
  Each SignOp type should be tested with a set of at least 100 randomized inputs for Instruction[25:0]
  Your testbench should create a VCD dumpfile for examination and debugging
- Copy your testbench code below.

You may leverage an LLM in generating this code, however you must ensure it does not have bugs and you will have to explain how it works during Lab.

Note: when I tried asking an LLM to do this (Fall'25), the result it produced had many bugs, make sure you understand what the code is doing and can correct it to perform the requested tasks. If you use an LLM, document below any errors you found, that you had to correct and how you corrected them.

Edit View Insert Format Tools Table

12pt Paragraph B I U A T²

p 0 words </>

Question 4 10 pts

What is the operation that your ALU performs with the following instructions?

LDUR - ADD

STUR - ADD

CBZ - pass input B

ORR - OR

Question 5 20 pts

Write the Verilog implementation of the ALU as specified in the instructions section at the top of this prelab. In particular, your implementation should conform to the following module definition:

```
`define AND   4'b0000  // These defines are not required but may be helpful.
`define OR    4'b0001
`define ADD   4'b0010
`define SUB   4'b0110
`define PassB 4'b0111

module ALU(
    output reg [63:0] BusW,
    input      [63:0] BusA,
    input      [63:0] BusB,
    input      [3:0]  ALUCtrl,
    output            Zero
);
```

Your Verilog implementation should conform to the following characteristics:

- It should be written in fully synthesizable behavioral Verilog with no unintended latches.
- Assume that ALUCtrl signal is encoded as described in the instructions for this prelab above.
- The Zero output should be set to 1 when the BusW output is all 0's, regardless of the type of operation performed.

Copy your Verilog code into the space below.

You may leverage an LLM in generating this code, however you must ensure it does not have bugs and you will have to explain how it works during Lab.

Edit View Insert Format Tools Table

12pt Paragraph B I U A T²

p 0 words </>

Question 6 30 pts

Write a self-checking testbench for your ALU.

Since the inputs and state elements are too many to exhaustively test, we will use a limited random testing strategy as we did in the previous lab. Thus, your testbench should perform tests which conform to the following constraints:

- The testbench should be self-checking, printing an error for each test that fails.
- Each of the functions of the ALU should be tested with a set of at least 100 randomized inputs for BusA and BusB
- Include at least one test which produces a Zero flag of 1 and one test that produces a Zero flag of 0 for each ALU function
- Include at least one test for the ALU that overflows during an ADD operation.
- Include tests where every combination of all ones and all zeros are used for each input for each of the functions of the ALU.
- Your testbench should create a VCD dumpfile for examination and debugging

Copy your testbench code below.

You may leverage an LLM in generating this code, however you must ensure it does not have bugs and you will have to explain how it works during Lab.

Note: when I tried asking an LLM to do this (Fall'25), the result it produced had many bugs, make sure you understand what the code is doing and can correct it to perform the requested tasks. If you use an LLM, document below any errors you found, that you had to correct and how you corrected them.

Edit View Insert Format Tools Table

12pt Paragraph B I U A T²

p 0 words </>

Question 7 10 pts

Fill in the truth table for your NextPC, what should the select on the mux be for these different inputs:

Branch	Uncondbranch	ALUZero	NextPC Mux Select
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1

1	1	1	1

Question 8 20 pts

Write the Verilog implementation of the NextPC unit as specified in the instructions section at the top of this prelab. In particular, your implementation should conform to the following module definition:

```
module NextPClogic(
    output reg [63:0] NextPC,
    input  [63:0] CurrentPC,
    input  [63:0] SignExtImm64,
    input         Branch,
    input         ALUZero,
    input         Uncondbranch
);
```

Your Verilog implementation should conform to the following characteristics:

- It should be written in fully synthesizable behavioral Verilog with no unintended latches.
- Assume that the sign extender does the left shifting of the SignExtImm64 as needed.
- Copy your Verilog code into the space below.

You may leverage an LLM in generating this code, however you must ensure it does not have bugs and you will have to explain how it works during Lab.

Edit View Insert Format Tools Table

12pt Paragraph B I U A T²

p 0 words </>

Question 9 20 pts

Write a self-checking testbench for your NextPC unit.

We will again use a limited random testing strategy as we did in the previous lab.

Your testbench should perform tests which conform to the following constraints:

- The testbench should be self-checking, printing an error for each test that fails.
- Your testbench should perform a set of 10 random data input (i.e CurrentPC and SignExtImm64) tests for each combination of the three control inputs.
- Your testbench should create a VCD dumpfile for examination and debugging

Copy your testbench code below.

You may leverage an LLM in generating this code, however you must ensure it does not have bugs and you will have to explain how it works during Lab.

Again, if you use an LLM and it produces errors, document below any errors you found, that you had to correct and how you corrected them.

Edit View Insert Format Tools Table

12pt Paragraph B I U A T²

p 0 words </>

Quiz saved at 11:56am Submit Quiz

